

OTEL-04: Trace Instrumentation

Series: OTEL — OpenTelemetry Integration | **Notebook:** 4 of 8 | **Created:** January 2026 | **Last Updated:** 10/02/2026

Instrumenting Applications for Distributed Tracing

Traces provide visibility into request flows across services. This notebook covers automatic and manual instrumentation techniques for popular languages with OpenTelemetry.

Table of Contents

- 1. [Instrumentation Types](#)
- 2. [Auto-Instrumentation](#)
- 3. [Manual Instrumentation](#)
- 4. [Context Propagation](#)
- 5. [Span Attributes and Events](#)
- 6. [Language-Specific Examples](#)
- 7. [Sampling: Head vs. Tail](#)
- 8. [Best Practices](#)

Prerequisites

Requirement	Details
Knowledge	OTEL-01 and OTEL-03

Requirement	Details
Environment	Collector deployed and accessible
Language	Examples in Python, Java, Go, Node.js

1. Instrumentation Types

Comparison

Type	Effort	Customization	Coverage
Zero-code (auto)	Minimal	Limited	Framework-level
Manual	More	Full	Custom business logic
Hybrid	Medium	Balanced	Best of both

When to Use Each

Scenario	Recommendation
Quick start, standard frameworks	Auto-instrumentation
Custom business spans	Manual
Production with specific needs	Hybrid
Serverless / Lambda	SDK with auto-instrumentation

Service Mesh: Istio & Envoy

A service mesh emits telemetry without touching application code — the sidecar proxy (Envoy) generates spans for the traffic it routes. This is zero-code instrumentation at the infrastructure layer, complementary to the SDK patterns above.

- **Envoy** can be configured to emit OpenTelemetry traces directly (an OpenTelemetry tracing provider) and ship them via OTLP — point the exporter at your collector or the Dynatrace OTLP endpoint.
- **Istio** wires this mesh-wide: set the mesh's tracing provider to OpenTelemetry and Istio applies the proxy tracing configuration across the mesh.

Mesh spans cover proxy-to-proxy hops; combine them with app-level SDK spans (via W3C trace-context propagation, §4) for an end-to-end trace. See the Dynatrace Istio/Envoy integration docs for the provider configuration.

Sources: [Istio and Envoy integration \(DT docs\)](#).

2. Auto-Instrumentation

Python Auto-Instrumentation

```
# Install
pip install opentelemetry-distro opentelemetry-exporter-otlp
opentelemetry-bootstrap -a install

# Run with auto-instrumentation
opentelemetry-instrument \
  --service_name my-python-app \
  --exporter_otlp_endpoint http://collector:4317 \
  python app.py
```

Java Auto-Instrumentation

```
# Download agent
curl -L -o opentelemetry-javaagent.jar \
  https://github.com/open-telemetry/opentelemetry-java-
instrumentation/releases/latest/download/opentelemetry-javaagent.jar

# Run with agent
java -javaagent:opentelemetry-javaagent.jar \
  -Dotel.service.name=my-java-app \
  -Dotel.exporter.otlp.endpoint=http://collector:4318 \
  -jar app.jar
```

Port 4318, not 4317. *"versions 2.0+ of the Java agent and OpenTelemetry Spring Boot starter use http/protobuf as the default protocol, not grpc"*, and OTLP/HTTP listens on 4318. Pointing a 2.x agent at the gRPC port 4317 fails to export. To use gRPC, also set `-Dotel.exporter.otlp.protocol=grpc`.

Sources: [Java agent configuration \(opentelemetry.io\)](https://opentelemetry.io/docs/quickstart/nodejs/).

Node.js Auto-Instrumentation

```
# Install
npm install @opentelemetry/auto-instrumentations-node \
  @opentelemetry/sdk-node \
  @opentelemetry/exporter-trace-otlp-http

# Run with auto-instrumentation
node --require @opentelemetry/auto-instrumentations-node/register app.js
```

```
// Or programmatically (tracing.js)
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-instrumentations-node');
const { OTLPTraceExporter } = require('@opentelemetry/exporter-trace-otlp-http');

const sdk = new NodeSDK({
  serviceName: 'my-node-app',
  traceExporter: new OTLPTraceExporter({
    url: 'http://collector:4318/v1/traces',
  }),
  instrumentations: [getNodeAutoInstrumentations()],
});

sdk.start();
```

3. Manual Instrumentation

Python Manual Tracing

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
OTLPSpanExporter
from opentelemetry.sdk.resources import Resource

# Setup
resource = Resource.create({"service.name": "my-python-app"})
provider = TracerProvider(resource=resource)
processor =
BatchSpanProcessor(OTLPSpanExporter(endpoint="http://collector:4317"))
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)

# Get tracer
tracer = trace.get_tracer(__name__)

# Create spans
@tracer.start_as_current_span("process_order")
def process_order(order_id):
    span = trace.get_current_span()
    span.set_attribute("order.id", order_id)

    with tracer.start_as_current_span("validate_order"):
        # Validation logic
        pass

    with tracer.start_as_current_span("charge_payment"):
        # Payment logic
        pass
```

Java Manual Tracing

```
import io.opentelemetry.api.GlobalOpenTelemetry;
import io.opentelemetry.api.trace.Span;
import io.opentelemetry.api.trace.Tracer;
import io.opentelemetry.context.Scope;

public class OrderService {
    private static final Tracer tracer = GlobalOpenTelemetry.getTracer("order-
service");

    public void processOrder(String orderId) {
        Span span = tracer.spanBuilder("process_order").startSpan();
        try (Scope scope = span.makeCurrent()) {
            span.setAttribute("order.id", orderId);

            validateOrder(orderId);
            chargePayment(orderId);
        } finally {
            span.end();
        }
    }
}
```

Go Manual Tracing

```
import (
    "context"
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
)

var tracer = otel.Tracer("order-service")

func ProcessOrder(ctx context.Context, orderID string) error {
    ctx, span := tracer.Start(ctx, "process_order")
    defer span.End()

    span.SetAttributes(attribute.String("order.id", orderID))

    if err := validateOrder(ctx, orderID); err != nil {
        span.RecordError(err)
        return err
    }

    return chargePayment(ctx, orderID)
}
```

4. Context Propagation

W3C Trace Context

The standard propagation format:

Header	Example	Description
traceparent	00-abc123-def456-01	Trace ID, span ID, flags
tracestate	dt=s:1	Vendor-specific state

tracestate is also how the browser session reaches the backend. Dynatrace's RUM JavaScript injects **traceparent** and **tracestate** on outbound requests, and the **tracestate** entry carries the RUM session identifier — **dt.rum.session.id** — alongside the trace and span IDs. Same propagation mechanism described above, used for a different join: it is what lets a backend span be attributed to the browser session that caused it, rather than only to its parent span.

On the span side that linkage surfaces as the `frontend.link` record (see **SPANS-04 § Sprint 1.344**). RUM-side configuration of it belongs to the **WEBRUM** series; the GenAI application — reconstructing a multi-turn LLM conversation from spans — is covered in **AIOPS-06 § 1**.

HTTP Propagation

Python (requests):

```
from opentelemetry.propagate import inject
import requests

def call_downstream():
    headers = {}
    inject(headers) # Adds traceparent, tracestate
    response = requests.get("http://downstream/api", headers=headers)
```

Extracting Context:

```
from opentelemetry.propagate import extract

@app.route('/api/endpoint')
def endpoint():
    context = extract(request.headers)
    with tracer.start_as_current_span("endpoint", context=context):
        # Request handling
        pass
```

Messaging Propagation

Kafka:

```
# Producer
from opentelemetry.propagate import inject

headers = []
inject(headers, setter=kafka_header_setter)
producer.send('topic', value=message, headers=headers)

# Consumer
context = extract(record.headers, getter=kafka_header_getter)
with tracer.start_as_current_span("process_message", context=context):
    process(record)
```


5. Span Attributes and Events

Adding Attributes

```
with tracer.start_as_current_span("checkout") as span:
    # Business attributes
    span.set_attribute("user.id", user_id)
    span.set_attribute("order.total", order_total)
    span.set_attribute("order.items", len(items))

    # Semantic convention attributes
    span.set_attribute("http.method", "POST")
    span.set_attribute("http.status_code", 200)
```

Adding Events

```
with tracer.start_as_current_span("payment") as span:
    span.add_event("payment_initiated", {"provider": "stripe"})

    result = process_payment()

    if result.success:
        span.add_event("payment_completed", {
            "transaction_id": result.transaction_id
        })
    else:
        span.add_event("payment_failed", {
            "error": result.error_message
        })
```

Recording Errors

```
from opentelemetry.trace import StatusCode

with tracer.start_as_current_span("operation") as span:
    try:
        result = risky_operation()
    except Exception as e:
        span.record_exception(e)
        span.set_status(StatusCode.ERROR, str(e))
        raise
```

Querying that status back out of Grail

The SDK constant and the Grail value are not the same string, and the mismatch is the single most common reason an error-rate query returns zero on a healthy pipeline:

Layer	Field	Value
OTel SDK (Python/Java/Go/JS)	<code>StatusCodes.ERROR</code>	<code>ERROR</code> — uppercase, this is the spec
OTel Collector config (<code>tail_sampling</code> <code>status_code</code> policy)	<code>status_codes: [ERROR, UNSET]</code>	<code>ERROR</code> — uppercase, also correct
Grail / DQL	<code>span.status_code</code>	<code>error</code> — lowercase

Two further Grail facts to carry into any DQL you write against span status:

- `otel.status_code` and `otel.status_message` **do not exist**. Neither has a row in `dt.semantic_dictionary.fields`. The real fields are `span.status_code` (`stable`) and `span.status_message` (`experimental` — and null on many failing spans, so do not depend on it for the failure reason).
- `span.status_code` **is null on successful spans**. `"ok"` is written vanishingly rarely — 9 spans out of roughly 659,000 in a two-hour window on a live tenant, 08/11/2026. So `countIf(span.status_code != "error")` counts almost nothing (a `!=` against null is null, not true). **Derive successes as `total - errors`**. An availability SLO built the other way reads 0% on a perfectly healthy service and reports no error while doing it.

Settle any such question against the dictionary rather than against a span sample — an empty span result is ambiguous, an empty dictionary result is not:

```
fetch dt.semantic_dictionary.fields
| filter startsWith(name, "span.status") or startsWith(name, "otel.status")
| fields name, stability, type
```

6. Language-Specific Examples

Complete Python Example

```
# app.py
from flask import Flask, request
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import
OTLPSpanExporter
from opentelemetry.instrumentation.flask import FlaskInstrumentor
from opentelemetry.instrumentation.requests import RequestsInstrumentor
from opentelemetry.sdk.resources import Resource

# Setup
resource = Resource.create({"service.name": "checkout-api"})
provider = TracerProvider(resource=resource)
processor = BatchSpanProcessor(
    OTLPSpanExporter(endpoint="http://collector:4318/v1/traces")
)
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)

# Auto-instrument
app = Flask(__name__)
FlaskInstrumentor().instrument_app(app)
RequestsInstrumentor().instrument()

tracer = trace.get_tracer(__name__)

@app.route('/checkout', methods=['POST'])
def checkout():
    with tracer.start_as_current_span("process_checkout") as span:
        order = request.json
        span.set_attribute("order.id", order['id'])

        # Custom business span
        with tracer.start_as_current_span("validate_inventory"):
            validate_inventory(order['items'])

    return {"status": "success"}
```

```
// View spans with custom attributes – order.id is an example; use an
attribute your code sets
fetch spans, from:-1h
| filter isNotNull(order.id)
| fields start_time, trace.id, span.name, order.id, duration
| sort start_time desc
| limit 20
```

```
// Find spans with errors
// The Grail field is span.status_code – NOT otel.status_code, which does not
exist
// (no row in dt.semantic_dictionary.fields). Likewise span.status_message,
not
// otel.status_message. Values are LOWERCASE in Grail: "error", never "ERROR".
fetch spans, from:-1h
| filter dt.openpipeline.source == "/api/v2/otlp/v1/traces" // OTLP-ingested
spans only
| filter span.status_code == "error"
| fields start_time, trace.id, span.id, span.name, span.kind,
span.status_message
| sort start_time desc
| limit 20
```

7. Sampling: Head vs. Tail

Sampling discards a portion of traces to control trace volume and cost. *Where* the decision is made changes what you can decide on.

Approach	Where	Decision basis	Trade-off
Head sampling	SDK, at span start	Random / rate-based only — the request outcome isn't known yet	Cheap and simple; keeps/drops blindly, so it can drop the errors and slow traces you most want
Tail sampling	Collector, after the trace completes	The full trace — status, latency, attributes	Keeps the interesting traces; costs memory/CPU (the collector buffers each trace until it finishes)

Head sampling (SDK)

```
from opentelemetry.sdk.trace.sampling import TraceIdRatioBased

# Keep 10% of traces — decided at the root span, propagated to children
provider = TracerProvider(sampler=TraceIdRatioBased(0.1))
```

Head sampling is consistent across a trace (the root decision propagates), but it cannot prefer errors or slow requests — it does not yet know how the request ends.

Tail sampling (Collector)

The `tail_sampling` processor buffers each trace until it completes, then applies policies — so you can keep every error and slow trace and probabilistically sample only the healthy remainder:

```
processors:
  tail_sampling:
    decision_wait: 30s          # how long to buffer a trace before deciding
    policies:
      - name: keep-errors
        type: status_code
        status_code: {status_codes: [ERROR, UNSET]}
      - name: keep-slow
        type: latency
        latency: {threshold_ms: 500}
      - name: sample-the-rest
        type: probabilistic
        probabilistic: {sampling_percentage: 20}
```

Tail sampling is **stateful** — *"All spans for a given trace MUST be received by the same collector instance for effective sampling decisions."* At scale that means two layers of collectors: a `loadbalancing` exporter in front — its default routing key for traces is `traceID` — and the tail-sampling tier behind it (the same two-tier shape OTEL-03 §5 uses for metrics, keyed by trace instead of resource). Never shard one trace across gateways.

The trap: sampling biases trace-derived metrics

If OpenTelemetry traces are sampled, trace-derived metrics (request counts, error rates) are computed only from the sampled subset — a 20% sampler makes throughput look 5× lower than reality.

Compute service metrics **before** sampling. The `span_metrics` connector (formerly `spanmetrics` — old name kept as a deprecated alias) reads the full, unsampled trace stream and emits RED metrics; `tail_sampling` then thins only the trace stream that gets stored:

```
connectors:
  span_metrics: {}
  forward/sampling: {}
service:
  pipelines:
    traces/in:                                # full stream
      receivers: [otlp]
      exporters: [span_metrics, forward/sampling]
    metrics/span_metrics:                     # metrics from ALL traces (unbiased)
      receivers: [span_metrics]
      exporters: [otlphttp]
    traces/sampled:                           # only sampled traces are stored
      receivers: [forward/sampling]
      processors: [tail_sampling]
      exporters: [otlphttp]
```

(Representative wiring — see the Dynatrace sampling docs demo for the full pipeline.)

Guidance

- **Do not mix sampling modes on one trace.** Combining OTEL sampling and OneAgent sampling on the same distributed trace yields inconsistent, partial traces — pick one owner per trace (see OTEL-07 §9 signal routing).
- Sampling is a **cost lever, not a default** — in community practice, teams start by keeping everything, measure ingest, and sample only if trace volume turns out to be the cost driver. OTEL trace volume is billed as Traces – Ingest & Process under DPS; only traces from a Full-Stack monitored host or the OneAgent Trace API draw on an included volume (see OTEL-01 §6 and the FINOPS series).

Sources:

~ [Sampling with the OTEL Collector \(DT docs\)](#) — `tail_sampling` policies

(`status_code` / `latency` / `probabilistic` , `decision_wait`), `span_metrics` -before-sampling to avoid metric bias, and the no-mixed-mode warning

~ [Tail Sampling Processor \(OpenTelemetry Collector Contrib GitHub\)](#) — “All spans for a given trace *MUST* be received by the same collector instance for effective sampling decisions.”

- [Load Balancing Exporter \(OpenTelemetry Collector Contrib GitHub\)](#) — "If no `routing_key` is configured, the default routing mechanism is `traceID` for traces"
- [OpenTelemetry licensing \(DT docs\)](#) — "OpenTelemetry trace data is billed by the ingested volume."

8. Best Practices

Span Naming

Good	Bad	Why
<code>HTTP GET /users/{id}</code>	<code>HTTP GET /users/123</code>	Low cardinality
<code>process_order</code>	<code>order_abc123</code>	Descriptive, stable
<code>db.query</code>	<code>SELECT * FROM...</code>	Operation, not data

Attribute Guidelines

Practice	Reason
Use semantic conventions	Standardization
Keep cardinality low	Query performance
Don't include PII	Security
Add business context	Debugging value
Prefer platform-native feature-flag enrichment where available	OneAgent 1.343 adds OpenFeature SDK support for Java — verify the agent version on the hosts concerned; hand-set flag-variant attributes remain the portable fallback (SPANS-02 § 7a)

Performance Tips

Tip	Impact
Use batch processor	Reduces network calls
Sample appropriately	Reduces volume

Tip	Impact
Limit attribute size	Reduces payload
Don't over-instrument	Reduces noise

Summary

In this notebook, you learned:

- Instrumentation types: auto, manual, hybrid
- Auto-instrumentation for Python, Java, Node.js
- Manual span creation and management
- Context propagation across services
- Adding attributes and events to spans
- Language-specific implementation patterns
- Best practices for effective tracing

References

- [OTel Python](#)
- [OTel Java](#)
- [OTel Go](#)
- [OTel Node.js](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.